

Homomorphic Encryption (HE) Basics

Compute on encrypted data while keeping inputs private. A foundational building block for privacy-preserving AI systems.



Why Homomorphic Encryption?

Sensitive data often needs machine learning processing:

- Health records
- Financial transactions
- Personal information

Traditional encryption requires **decryption before use** → creates security vulnerabilities

The HE Advantage

Compute **directly on ciphertexts** without exposing raw data

Result remains encrypted and can only be decrypted by the key holder



The Core Idea

Normally: `Enc(x)` hides x , but prevents computation

With HE, encryption preserves mathematical relationships:

$$Enc(x) \oplus Enc(y) = Enc(x + y)$$

$$Enc(x) \otimes Enc(y) = Enc(x \times y)$$



This means we can add or multiply encrypted numbers, and the results **stay encrypted** while preserving the correct mathematical relationship.

Types of Homomorphic Encryption

1

Partially Homomorphic (PHE)

Supports *either* addition *or* multiplication (not both)

Examples: RSA (multiplication), Paillier (addition)

2

Somewhat Homomorphic (SHE)

Supports limited number of operations before noise makes results unusable

Practical for specific use cases with bounded computation

3

Fully Homomorphic Encryption (FHE)

Supports unlimited operations (both addition and multiplication)

First practical scheme by Craig Gentry (2009)

Enables arbitrary computation on encrypted data

Why FHE is Revolutionary

Enables arbitrary ML inference on encrypted data without revealing inputs

Real-world example:

1. Hospital encrypts sensitive patient data
2. Cloud service runs ML model → produces encrypted diagnosis
3. Only hospital can decrypt and view results

The cloud provider **never sees raw data** → true privacy



Simple Example: Addition with Paillier

```
from phe import paillier

# Generate keys
public_key, private_key = paillier.generate_paillier_keypair()

# Encrypt numbers
x, y = 5, 7
enc_x, enc_y = public_key.encrypt(x), public_key.encrypt(y)

# Homomorphic addition
enc_sum = enc_x + enc_y

# Decrypt result
print(private_key.decrypt(enc_sum)) # Output: 12
```

✓ **Key insight:** The computation happened entirely on encrypted values!

This simple example demonstrates homomorphic addition with the Paillier cryptosystem.

Even though we performed `enc_x + enc_y`, neither the operation nor the cloud service ever saw the raw values 5 and 7.

Challenges of Homomorphic Encryption

Performance

Operations are $\sim 1000\times$ slower than plaintext computation

Complex ML inference can take minutes instead of milliseconds

Memory Overhead

Ciphertexts are significantly larger than plaintexts

Can increase memory requirements by $10-1000\times$

Implementation Complexity

Deep learning operations need mapping to HE primitives

Non-linear functions (ReLU, sigmoid) are especially challenging

Maturity

Still evolving technology

Not yet widely deployed in production AI systems

HE in AI Infrastructure



Inference on Encrypted Data

Models process sensitive healthcare and financial information without exposure



Encrypted Feature Extraction

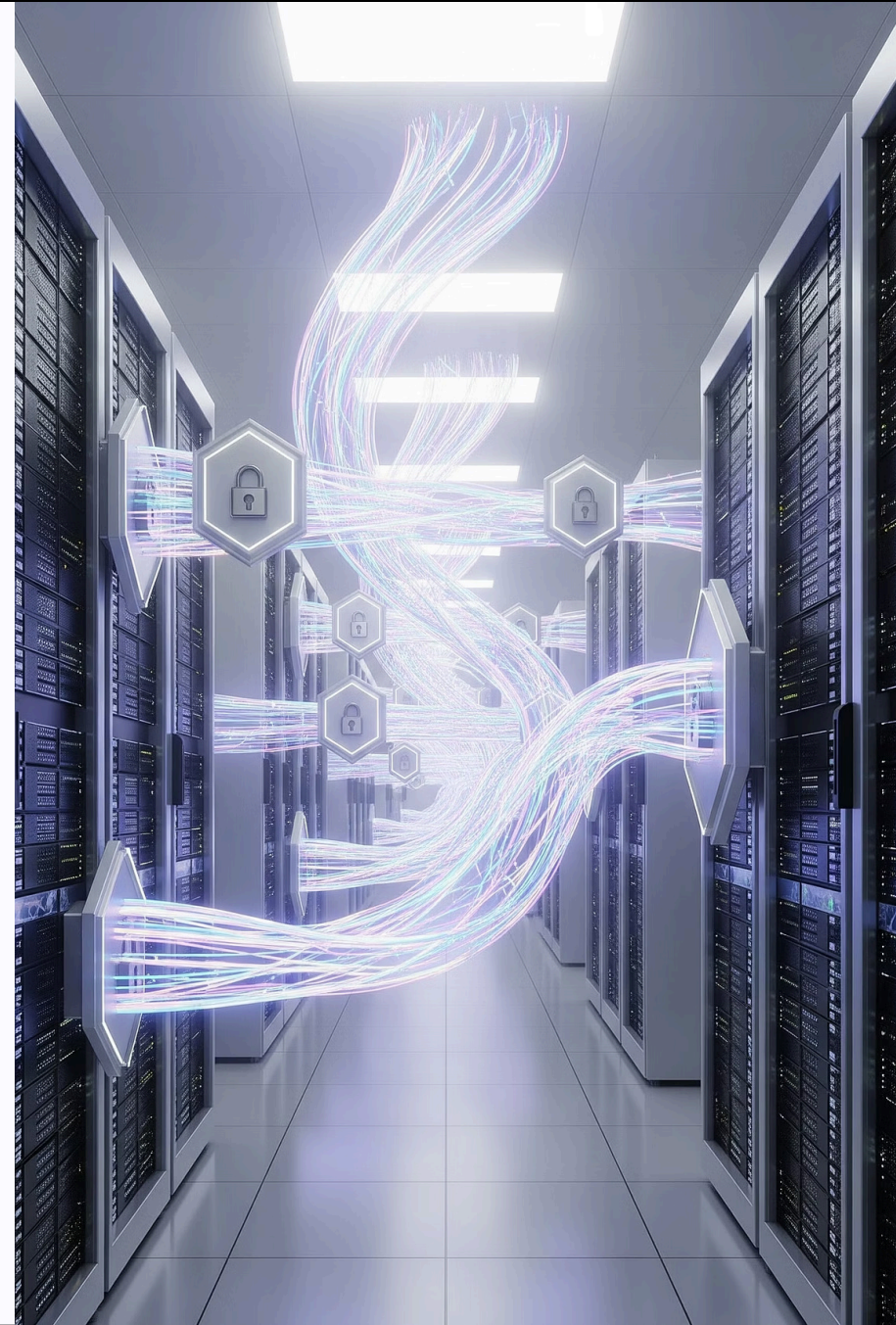
Privacy-preserving preprocessing in ML pipelines



Hybrid Privacy Systems

Combines with Federated Learning and Differential Privacy for layered protection

Integration into libraries accelerating adoption: Microsoft SEAL, IBM HELib, TenSEAL



Industry Applications



Healthcare

Hospitals collaborate on encrypted patient data for improved diagnostics while maintaining privacy



Finance

Cross-institutional fraud detection without revealing sensitive transaction details



Cloud AI

Inference-as-a-service without exposing customer data to cloud providers



Government

Privacy-preserving census analysis and defense applications

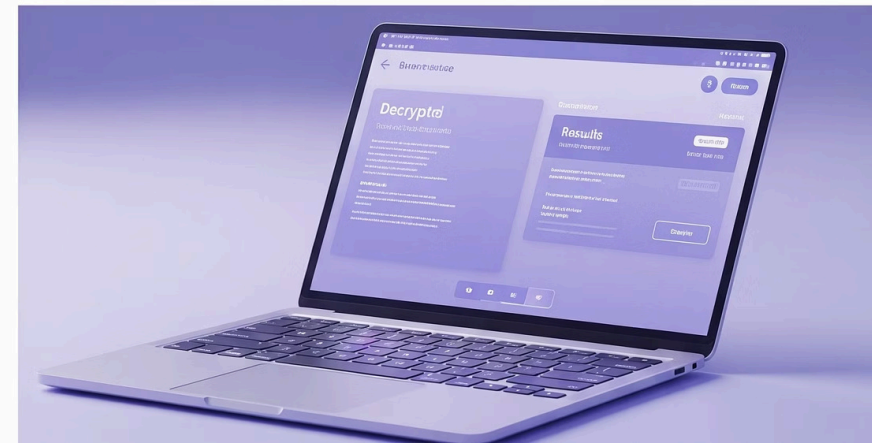
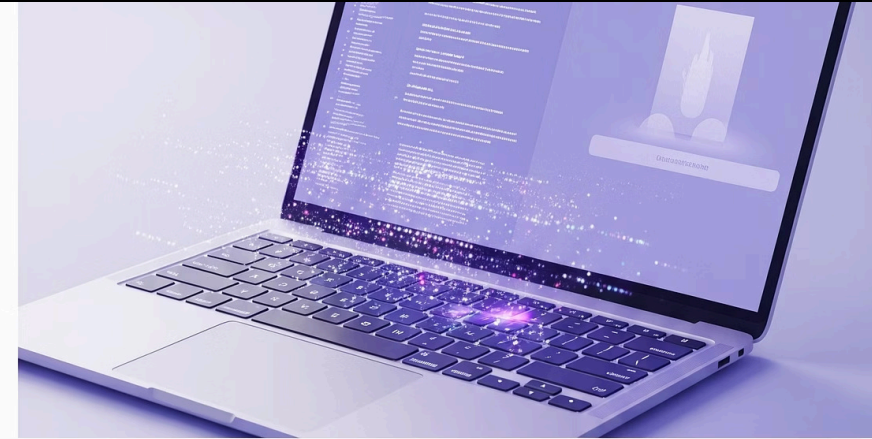
How HE Works in Production

Client Side

1. Generate encryption keys
2. Encrypt sensitive data
3. Send ciphertext to server
4. Decrypt results with private key

Server Side

1. Receive encrypted data
2. Run ML models on ciphertexts
3. Return encrypted results
4. Never sees raw data



Future Directions



Hardware Acceleration

GPU, FPGA, and ASIC implementations for FHE to dramatically reduce computational overhead



Optimized Schemes

CKKS for approximate real numbers, specialized implementations for common ML operations



Hybrid Privacy Stack

Integrated solutions combining Federated Learning, Differential Privacy, and Homomorphic Encryption



Practical Deep Learning

Making HE feasible for complex neural network inference at scale



Summary: Homomorphic Encryption for AI

Key Benefits:

- Compute directly on encrypted data
- No need to decrypt during processing
- Enables privacy-preserving ML inference
- Essential for sensitive AI domains

Current Limitations:

- Performance overhead
- Implementation complexity
- Scalability challenges
- Active area of research

As HE techniques mature, they will become a critical component in the privacy-preserving AI toolkit, especially for healthcare, finance, and government applications.